Deliverable 0 — Technical Onboarding & Product Exploration

CourtIQ · ADSC Fall 2026 · Individual assignment for all nine members

Time: approximately 1–2 hours (about 105 minutes: setup 25, notebook 45, proposal 20, submission 15). Installation/network problems may take longer; ask a PM if blocked. **Due:** before the first team meeting; the PM will place the exact date/time in the shared GitHub Issue. Each member submits their own work even when discussing ideas with teammates.

Learning objectives

By the end, you can use one Python environment, inspect and filter a pandas table, distinguish season totals from per-game averages, create an interactive Plotly chart, explain a result's limitations, and submit work through the team's feature-to-integration PR workflow.

Part A — Development environment (25 minutes)

Follow the full Windows/macOS/Linux setup guide. After cloning, use submission steps 1–2 below to create your member worktree, then set up the environment there. Create and activate `.venv` with Python 3.12, install `requirements.txt`, then run from the repository root:

```
python scripts/verify_environment.py
python -m pip check
python -m ipykernel install --sys-prefix --name courtiq --display-name "CourtIQ (Python 3.12)"
python -m notebook
```

Select the CourtIQ kernel. Check `sys.executable` in a notebook cell: it should point into `.venv`. Run the starter imports and data-loading cell. Capture the verifier's PASS and successful notebook/figure execution for your PR; hide local tokens and private paths if necessary. The Streamlit shell is also available with `python -m streamlit run app.py`; explore its four sections to inform Part C.

For failures, use the setup guide's troubleshooting table. Tell a PM the command, error message, OS and Python version; never send tokens or credentials. Live NBA requests are not needed.

Part B — Guided NBA-style data exploration (45 minutes)

First create your individual notebook copy using the submission instructions below. Open it in Jupyter. The supplied synthetic fixture contains **12 fictional NBA-style player rows, not real NBA results**. Names, teams and seasons are invented. The fixture dictionary explains every column.

Work through the clearly labeled notebook TODOs:

1. Load the fixture using the supplied code and inspect its first rows, dimensions, types and columns.
2. Explain `GP`, `PTS_TOTAL` and `PTS_PER_GAME`. Totals accumulate across games; per-game values are already averaged. **Do not divide `PTS_PER_GAME` by `GP` again.** For a separate example, 120 total points over 10 games means 12 points per game; a stored average of 12 remains 12.
3. Keep players with **at least 20 games**, including exactly 20. Investigate how low-game players could distort a ranking.
4. Rank the eligible rows by **points per game descending**, and keep the top five. Preserve that order for the chart. Use averages, not total points.
5. Create a Plotly bar chart: player name on x, points per game on y, highest to lowest left to right. Label the title as **synthetic** and the axes with readable names/units. Show the chart interactively in the notebook.
6. Write 3–5 sentences interpreting the ranking, the impact of the 20-game rule and at least one limitation. Do not generalize fictional data to the actual NBA or infer overall ability from scoring alone.

Hints: look up pandas boolean indexing, `sort_values`, `head`, and Plotly Express `bar`. For an unrelated example of filtering, `practice[practice["AGE"] >= 18]` keeps adults. The exercise's actual filtering/ranking/chart code is yours to write. Starter `None` values let unanswered cells show reminders; replace them, then restart the kernel and **Run All**. Unanswered TODOs are not completed work even if Run All has no errors.

Check yourself: every ranked player has at least 20 games; there are five distinct rows; their averages descend; the chart uses those same rows and values. You can use `PTS_TOTAL / GP` to confirm the stored average, but do not replace an average with an average divided again.

Optional extension (not required or graded): Read the nba_api endpoint documentation and identify a potential endpoint such as `LeagueDashPlayerStats` or `PlayerGameLog`. Discuss its fields and per-mode parameters. Actual retrieval requires PM confirmation of source permission per DATA.md; requests may time out or be blocked. Keep any approved downloads under ignored `data/`, never in a PR. This assignment works entirely offline after package installation.

Part C — Product brainstorming (20 minutes)

Copy the proposal template to your individual path below. Write about 150–250 words and optionally include a linked example or sketch. Answer all five questions: what you propose, the fan's question, required data, how it might work in CourtIQ, and a relevant example/sketch/visualization (or describe an original sketch in words).

Permanent team	Members	Initial research focus
A — Data and Architecture	Aiden Jackson, Ignacio Wijatno, Andrew Mok	NBA data availability, useful endpoints, necessary fields and data organization
B — Application and Visualization	June Hwang, Jason Q. Koffi, Sri Abhinav Thatavarthi	Player/team layouts, Streamlit interactions and effective basketball charts
C — Statistics and Analytics	Michael Du, Casey Hawkings, Jack Dunlap	Useful statistics, comparisons, percentiles and recent-performance methods

These are temporary research responsibilities. Teams remain together but rotate technical responsibilities during the semester. Discuss with teammates, but each member submits one useful proposal. Proposals support our first meeting's design discussion; they do not automatically expand scope. See the application plan for the four planned sections.

GitHub submission (15 minutes)

Follow the established contribution workflow. The commands below are for **members submitting their assignment**, not authorization for an agent to publish this starter.

1. Find the issue and integration branch

Open the repository's GitHub **Issues** tab and find **Deliverable 0 — Technical Onboarding & Product Exploration**. Read its deadline/checklist, then comment with your GitHub username and permanent team to claim your individual slice. This is one shared issue with nine submissions; do not claim the entire issue exclusively. The PM creates/publishes `issue-<n>/integration` with the starter files before members begin. If it does not exist or the issue is missing, ask a PM rather than inventing a number or targeting `main`.

2. Create your feature worktree

From your clean clone's root, run `git fetch origin`. In the following example, **replace `42` with the actual issue number** and **replace `your-handle` with your lowercase GitHub username** everywhere. Do not type the placeholder strings unchanged.

```
git fetch origin
git worktree add -b issue-42/onboarding-your-handle ../courtiq-onboarding-your-handle origin/issue-42/
    integration
cd ../courtiq-onboarding-your-handle
```

This follows the flat `issue-<n>/<slice>` naming rule. In the new worktree create/activate `.venv` and install dependencies using SETUP.md; use that same environment for all tools in this checkout. If you already did Part A in the initial clone, repeat environment setup here before notebook work. To avoid duplicate installation, create this worktree immediately after cloning and do Part A there.

3. Copy your individual files

From the activated terminal at your worktree root, run `python` to open the Python prompt. Replace `your-handle` in the snippet, then paste these lines (without typing the `>>>` prompt):

```
from pathlib import Path
handle = "your-handle"
nb = Path("notebooks/submissions") / f"{handle}_00_onboarding.ipynb"
proposal = Path("docs/proposals") / f"{handle}.md"
nb.parent.mkdir(parents=True, exist_ok=True)
proposal.parent.mkdir(parents=True, exist_ok=True)
with nb.open("xb") as output:
    output.write(Path("notebooks/00_onboarding.ipynb").read_bytes())

with proposal.open("xb") as output:
    output.write(Path("docs/FEATURE_PROPOSAL_TEMPLATE.md").read_bytes())

exit()
```

Keep the blank lines after each indented block when pasting. Exclusive `"xb"` mode prevents overwriting existing copies. If you see `FileExistsError`, open your existing files instead of copying again.

Replace `your-handle` first. Complete `notebooks/submissions/your-handle_00_onboarding.ipynb` and `docs/proposals/your-handle.md`. **Do not edit the shared starter notebook or fixture.** Keep the notebook inside the checkout so its relative fixture lookup works. In a proposal copy, resource links should use full URLs or paths relative to `docs/proposals/`. Screenshots can be attached directly to the PR; they do not require shared tracked filenames.

4. Save, check and commit your own work

Restart the notebook kernel and Run All; confirm imports, filtering, top five and the interactive chart work. Save with useful outputs visible; inspect them for private paths/tokens. From the worktree root:

```
git status --short
git diff --check
git add notebooks/submissions/your-handle_00_onboarding.ipynb docs/proposals/your-handle.md
git diff --cached --stat
git commit -m "Complete individual CourtIQ onboarding exercise and proposal"
git push -u origin issue-42/onboarding-your-handle
```

Stage only your two files. Do not commit datasets downloaded from NBA sources, `.venv`, credentials or caches. Our synthetic fixture is already supplied; it should not appear as your change.

5. Open the correct pull request and request review

On GitHub choose **Pull requests → New pull request**, set **base** to `issue-42/integration` and **compare** to `issue-42/onboarding-your-handle`. Confirm your PR changes only your files. Title: `Deliverable 0: Your Name`. Describe:

- Your name, username and team; `Related to #42` (use the actual number).
- OS, Python version, verifier result and kernel confirmation.
- Notebook/chart screenshot and a short account of what you learned.
- Link to your proposal; any installation issue or remaining question.

Use **Related to**, rather than **Closes**, so one member's submission does not close the shared issue. Request a teammate's or PM's review in GitHub's Reviewers control. Wait for any configured automated review and human review. Read all feedback, edit your two files, rerun affected cells, save, commit and push follow-up changes, then re-request review. Merge only after approval following CONTRIBUTING. The PM owns the eventual integration-to-`main` review.

Acceptance checklist for each member

- ☐ Python 3.12 and one `.venv`; verifier PASS and `pip check` successful.
- ☐ Notebook uses the CourtIQ kernel and loads the supplied fixture without live API access.
- ☐ All TODOs answered, including inspection and explanation of totals versus averages.
- ☐ Filter includes exactly 20 games and excludes every row below 20.
- ☐ Five eligible players ranked by points per game descending.
- ☐ Interactive Plotly chart uses the same five rows, columns and order, with synthetic labeling.
- ☐ 3–5 sentence interpretation discusses the threshold and a limitation.
- ☐ Complete feature proposal with all questions answered and team research focus.
- ☐ Unique notebook/proposal paths, execution evidence, correct integration-base PR and requested review.
- ☐ No secrets, real downloaded data, virtual environments or cache artifacts submitted.

Useful resources

- Python venv documentation
- pandas getting started
- Plotly bar charts
- Jupyter Notebook documentation
- Streamlit getting started
- GitHub pull requests

The Markdown assignment is authoritative. A shareable PDF accompanies this version; use the repository links for code and the latest instructions.